💼 Getting Started with Ansible: Automate Your Infrastructure

🔍 **What is Ansible?**
Ansible is an open-source automation engine used for:

🔧 **Configuration Management** – Install packages, manage services
🚀 **Application Deployment** – Deploy Node.js, Java, Python apps
☁️ **Cloud Provisioning** – Automate AWS, Azure, GCP, OpenStack
📊 **Legacy System Automation** – Manage old systems without agents

✨ **Key Features:**
✅ Agentless (via SSH/WinRM)
✅ Written in Python
✅ Uses YAML
✅ Idempotent (safe to re-run)
✅ Push model (Ansible Pull optional)

💡 **Why not Shell Scripts?**
Shell scripts often fail to scale due to:

- ❌ Non-idempotent behaviour
- ❌ OS-specific commands
- ❌ Hard-to-debug tangled logic

Ansible gives you:
✅ Idempotency through modules
✅ Conditional execution & retries
✅ Structured syntax that supports modules, loops, handlers, and logging

---

🔧 **Getting Started:** Installing Ansible

Ubuntu/Debian:
📦 sudo apt install ansible -y

RHEL/CentOS:
📦 sudo dnf install ansible -y

Test connection:
🔗 ansible -i <ip>, all -e ansible_user=ec2-user -e ansible_password=yourpass -m ping

---

🗂️ **Inventories**
Define which nodes to manage.

Static Inventory:
[web]
192.168.1.10

[db]
192.168.1.20

Test:
ansible all -i inventory.ini -m ping -u ec2-user --ask-pass

Supports:

- Grouped/ungrouped/nested hosts
- Per-host/group variables
- Host patterns (e.g., web & db)

⚡ **Ad-Hoc Commands**
Quick one-liners for troubleshooting or small tasks:

```
ansible web -m shell -a "uptime"
ansible db -m yum -a "name=httpd state=present"
```

Use for:
Restarting services
Debugging
Patching
System checks

📜 **Playbooks:** Declarative YAML files that define your automation tasks.

```
- name: Setup Web Server
  hosts: web
  become: yes
  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present
```

🧱 **Modules:** Reusable Task Units
Ansible comes with hundreds of built-in modules to manage everything.

Common Modules:
ansible.builtin.ping — check if host is reachable
ansible.builtin.package — install or remove software
ansible.builtin.service — start/stop/restart services
ansible.builtin.user — create or manage users

🆚 **Shell vs Command**

```
- name: Using shell
  ansible.builtin.shell: echo $HOME > /tmp/home.txt

- name: Using command
  ansible.builtin.command: echo hello
```

✔ command is safer — avoids variable interpolation
✔ shell is more flexible — use pipes, redirects, variables

🔣 **Data types, Variables**
✅ **Data Types:**
String
Integer,
Boolean
List
Dictionary

Variables:
Define dynamic values.

```
vars:
  user: renuka
  port: 8080
  is_active: true
  packages: ["nginx", "git"]
  config: {mode: "production", debug: false}
```

Use in tasks:

```
- debug:
    msg: "{{ user }}"
```

📩 **Variable Injection**

✅ **From CLI:**

```
ansible-playbook site.yml -e "username=ram"
```

✅ **Prompt user input:**

```
vars_prompt:
  - name: username
    prompt: "Enter your name"
    private: no
```

✅ **From file:**

```
vars_files:
  - vars.yml
```

🔁 **Loops**
Avoid repetitive tasks.

```
- name: Install tools
  ansible.builtin.package:
    name: "{{ item }}"
    state: present

  loop:
    - git
    - curl
    - unzip
```

⚠️ **Conditions (when):** Run tasks conditionally

```
- name: Install nginx on RHEL
  ansible.builtin.yum:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == "RedHat"
```

🧠 **Filters & Jinja2:** Modify or format variables.

```
msg: "{{ username | default('admin') | upper }}"
```

✅ **Useful filters:** default, upper, join, replace, length, unique

---

🧠 **Facts:** Auto-gather system info
Ansible automatically collects facts about the target machines at the beginning of playbook execution.

```
- debug:
  msg: "{{ ansible_facts['distribution'] }}"
```

To disable it:
gather_facts: false

Use for:
OS detection
IP, memory, CPU discovery
Logic branching

🔐 **Ansible Vault:** Secure secrets.

```
ansible-vault create secrets.yml
```

Use in playbook:

```
vars_files:
 - secrets.yml
```

Run:
ansible-playbook site.yml --ask-vault-pass

🧱 **Roles:** Reusable Components
Split playbooks into reusable units.

Structure:

```
roles/
└── nginx/
    ├── tasks/main.yml
    ├── handlers/main.yml
    ├── templates/nginx.conf.j2
    ├── vars/main.yml
    ├── defaults/main.yml
```

Use in playbook:

```
roles:
 - nginx
```

✅ Encourages reuse
✅ Clean structure
✅ Team collaboration

🏷️ **Tags:** Run only part of a playbook.

```
- name: Install NodeJS
 tags: node
 ansible.builtin.package:
  name: nodejs
  state: present
```

Run with:
ansible-playbook site.yml --tags node

🔔 **Handlers:** Trigger actions on change.

```
- name: Copy config
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx

handlers:
  - name: Restart nginx
    ansible.builtin.service:
      name: nginx
      state: restarted
```

☁️ **Dynamic Inventory**
Auto-discover cloud infrastructure.

AWS Example:

```
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters:
  instance-state-name: running
```

List inventory:

```
ansible-inventory -i aws_ec2.yml --graph
```

Supports: AWS, Azure, GCP, VMware, K8s

❗ **Error Handling**
Gracefully handle task failures using ignore_errors: yes

```
- name: Check if user exists
  command: id deploy
  register: result
  ignore_errors: yes

- name: Create user
  ansible.builtin.user:
    name: deploy
  when: result.rc != 0
```

Try/Catch:

```
block:
  - name: Try something
    ...
rescue:
  - name: Handle failure
    ...
always:
  - name: Cleanup
    ...
```

⚙️ **Performance & Control**
Optimize execution across nodes.

📌 Parallelism – run tasks concurrently:

```
forks = 10                    # 10 tasks in parallel
```

📌 Rolling Updates – update a few nodes at a time:

```
serial: 3                     # 3 hosts at a time
```

🏢 **Ansible Tower (Red Hat AAP)**
Ansible Tower adds enterprise features.

🖥️ Web Dashboard
🔐 RBAC (Role-Based Access)
📅 Job Scheduling
📁 Git Integration
📄 Job Templates
📊 Audit Trails
🔁 Workflow Automation

📘 **Tower in Action: Real-World Use Case**

Manage 500+ servers with a click:
Open Tower UI
Launch "Deploy App" job template
Pass vault credentials
Watch real-time logs
Check historical reports

Example Job Template Playbook:

```
- name: Deploy Web App
  hosts: web
  become: yes
  vars_files:
    - secrets.yml
  roles:
    - app_deploy
```

🔄 **Git + Tower = CI/CD Power**
Push code to GitHub → Tower auto-triggers deployment

✅ Zero manual intervention
✅ Integrated testing
✅ Fast, reliable deployments